Module 13 — Scale & LM Deployment

Fine-Tuning DistilBERT for Grad Cafe Admissions Prediction

Preprocessing Decisions

The dataset (cleaned_gradcafe.json, an already-cleaned version of the raw Grad Cafe scrape used in earlier modules) was loaded into a Pandas DataFrame, filtered to rows with a status of Accepted or Rejected, deduplicated by result URL, and required to have a usable llm-generated-program value. Text fields (llm-generated-program, llm-generated-university, comments, term) and categorical fields (Degree, US/International) were normalized so any missing, null, or empty value became the literal string "Unknown", applied consistently everywhere. GPA, GRE, GRE V, and GRE AW were converted to numeric types via pandas.to_numeric with invalid values coerced to NaN.

Metric	Value
Rows in original dataset	29,831
Rows remaining after filtering	25,077
Accepted rows	11,050
Rejected rows	14,027
Text fields used	llm-generated-program, llm-generated-university, comments, term
Non-text fields used	Degree, US/International, GPA, GRE, GRE V, GRE AW

Model Input Template

Every applicant (at training time and at inference time from the Flask form) is converted into the exact same labeled text block before tokenization, defined once in model_common.py and imported by both train_model.py and inference.py so the training and serving formats can never drift apart. Numeric values are formatted with Python's general float format (e.g. GPA 3.90 and the string "3.90" both render as "3.9"), so a value entered differently on the webpage than it appeared in training data still produces identical model input text. The label is never included in this text.

```
Program: {program}
University: {university}
Comments: {comments}
Term: {term}
Degree: {degree}
Citizenship: {citizenship}
GPA: {gpa}
GRE: {gre}
GRE Verbal: {gre_v}
GRE AW: {gre_aw}
```

Sample model input (from the training dataset, label = Rejected):

```
Program: Statistics
University: University of Wisconsin-Madison
Comments: Unknown
```

```
Term: Fall 2026
Degree: PhD
Citizenship: International
GPA: 3.72
GRE: Unknown
GRE Verbal: Unknown
GRE AW: Unknown
```

Training Configuration

Setting	Value
Model	distilbert-base-uncased
Tokenizer	distilbert-base-uncased (matching WordPiece tokenizer)
Max sequence length	256
Batch size	8
Epochs	5
Learning rate	2e-5
Optimizer	AdamW
Device	CUDA (GPU)

Tokenizer choice: DistilBERT's WordPiece tokenizer is used because it exactly matches the pretrained checkpoint's vocabulary. DistilBERT retains roughly 97% of BERT's language understanding while being about 40% smaller and 60% faster, making it (and its matching tokenizer) the recommended choice for fine-tuning on ordinary hardware, per the assignment guidance.

Training ran for 5 epochs on the full 20,061-row training set (2,508 steps per epoch) on a GPU.

Evaluation Metrics

Metric	Value
Accuracy	78.95%
Precision	76.40%
Recall	75.57%
F1 score	75.98%
Test set size	5,016

Confusion matrix (rows = actual, columns = predicted):

	Predicted Rejected	Predicted Accepted
Actual Rejected	2,290	516
Actual Accepted	540	1,670

Class distribution: the test set contained 2,806 actual Rejected and 2,210 actual Accepted applicants (55.9% / 44.1%, matching the training set's split). The model predicted 2,830 Rejected and 2,186 Accepted, close to the true distribution.

Is the model biased toward one class? Not substantially. Both false-positive (516) and false-negative (540) counts are of similar magnitude, and the predicted class totals (2,830 / 2,186) are close to the actual totals (2,806 / 2,210) rather than skewed sharply toward over-predicting either outcome, though the model leans slightly more toward predicting Rejected than the true distribution would suggest.

Is performance meaningfully stronger than random guessing? Yes. 78.95% accuracy clears both a coin-flip baseline (50%) and the dataset's own majority-class baseline (55.9%, always predicting Rejected) by a wide margin.

Is the dataset sufficient for a realistic admissions predictor? Only partially. GRE-related fields remain missing for the large majority of applicants, GPA is missing for a substantial share, and the dataset has no access to letters of recommendation, funding availability, or interview outcomes, all of which plausibly matter in real admissions decisions but are entirely absent here.

Comparison to the Module 12 Two-Layer Network

The DistilBERT model reaches 78.95% test accuracy, a meaningful improvement (about 9.5 percentage points) over the Module 12 hand-built NumPy network's 69.48%, using the same underlying applicant pool. The transformer can use information the two-layer network structurally could not: free-text comments and program/university names, processed through a pretrained language model that already understands English semantics, rather than being limited to six hand-picked numeric and binary features. Whether the added text specifically helped, versus the transformer's greater raw capacity, cannot be fully separated without an ablation run that removes the text fields and retrains, but the comments field in particular (which the two-layer network never saw) plausibly carries real signal, such as mentions of research experience, publications, or self-assessed application strength.

The pretrained model is more flexible in what kinds of information it can absorb (arbitrary text, not just fixed numeric columns), but also more fragile in other ways: it is far more expensive to train (minutes on a GPU versus seconds for the NumPy network), requires downloading a large pretrained checkpoint, and its behavior is much harder to inspect or explain than six weights feeding one hidden layer. Given the accuracy improvement is real but not dramatic (roughly 9 percentage points), the added complexity is only modestly justified by the results here. It is a legitimate improvement, not a transformative one.

Reflection on Limitations and Ethics

What is useful about fine-tuning a pretrained model rather than training from scratch?

A pretrained language model already encodes a broad understanding of English before it ever sees a single Grad Cafe row, so fine-tuning only needs to adapt that existing understanding to this narrower task rather than learn language from nothing. That is why 20,061 training examples and a handful of epochs are enough to reach roughly 79% accuracy here, whereas training a comparably capable text model from scratch would need vastly more data and compute.

Why is combining text and structured features interesting for this task?

Structured fields like GPA and GRE are precise but narrow, while free-text comments can carry context no numeric field captures, such as research experience, funding concerns, or an applicant's own read on their chances. Combining both lets the model draw on whichever signal is actually present for a given applicant, rather than being limited to only one type of information.

What kinds of bias may exist in self-reported Grad Cafe data?

Applicants who post to Grad Cafe are a self-selected group, likely skewed toward people anxious enough about admissions to seek out and post on a public forum, and outcomes are self-reported with no independent verification. Posting behavior may also differ systematically between accepted and rejected applicants, and between applicants from different backgrounds, in ways that would bias which outcomes and which language patterns the model actually learns from.

Why might this model produce misleading or unfair predictions?

The model can only reproduce patterns already present in scraped, self-reported data, including any of the biases above, and has no way to represent the specific, holistic judgment an actual admissions committee applies to a given file. A model that appears roughly 79% accurate in aggregate can still be systematically wrong for individuals whose profile does not resemble the patterns it learned from, without that being visible from the overall metrics alone.

What important information about real admissions decisions is missing?

Letters of recommendation, the actual statement of purpose (only informal comments are available here), interview performance, funding availability, and program-specific priorities in a given admissions cycle are all absent from this dataset, despite plausibly mattering as much as or more than GPA and test scores in real decisions.

Should a model like this ever be used in real decision-making?

No. It was trained on unverified, self-reported, incomplete data for a course assignment, not validated against real institutional criteria, and cannot account for the specific, holistic judgment real admissions committees apply. Using it, or a model like it, to actually inform an admissions decision would risk real harm to applicants based on patterns that may reflect dataset bias rather than genuine merit.

How should uncertainty and user trust be handled on a public-facing prediction page?

The page should state plainly, and prominently, that it is a class project and not a real admissions tool, show the model's confidence score rather than only a binary label so users can see how uncertain a given prediction is, and avoid any language that could be read as authoritative or institutional. The deployed page here does exactly this: a disclaimer banner above the form, and both the predicted label and the underlying probability displayed together in the result.

Why is a project like this educationally valuable despite its limitations?

Building this pipeline end to end, from raw scraped data through a fine-tuned transformer to a working, publicly-reachable prediction page, is genuinely useful practice for the full shape of a real applied machine learning system: data cleaning, feature representation, model training, evaluation, serialization, and deployment. The value is in learning that pipeline, not in the specific predictions the resulting model happens to produce, which is exactly why the disclaimer and the honest limitations discussed above matter as much as the model itself.